

Name: Su Chuanzhou
UID: 3036837980

COMP7508 Assignment 1 Report

Task 1 Character Animation

I created a character animation in Blender in which the character jumps and waves. I used a separate action named `my-fk-animation`, set key poses for the movement, and adjusted their timing so the motion reads as one repeatable clip. I rendered three repetitions into the requested six-second, 24-fps video.

The frames below show representative poses from the rendered clip. The full video is “Rendered video.mp4”.



Figure 1. Starting pose.



Figure 2. Raised-arm pose during the jump-and-wave action.



Figure 3. Return pose.

Task 2 Forward Kinematics

I filled in the missing T-pose and forward-kinematics code in the designated regions of `task2_forward_kinematic.py`. I then ran the viewer on `motion_walking.bvh` and checked both the reconstructed T-pose and the animated walking motion. For the T-pose, I accumulated each joint's offset from its parent. Because the joint list is in parent-before-child order, one pass is enough:

```
offset = np.asarray(joint_offsets[joint_idx]).reshape(3)
if parent_idx == -1:
    global_joint_position[joint_idx] = offset
else:
    global_joint_position[joint_idx] = (
        global_joint_position[parent_idx] + offset)
```

For each BVH frame, I transformed a child position by its parent's global rotation, then composed the parent and local rotations. This is the core of the FK implementation:

```

if parent_idx == -1:
    global_joint_positions[:, joint_idx, :] = joint_positions[:, joint_idx, :]
    global_joint_orientations[:, joint_idx, :] = joint_rotations[:, joint_idx, :]
else:
    parent_rotation = R.from_quat(
        global_joint_orientations[:, parent_idx, :])
    global_joint_positions[:, joint_idx, :] = (
        global_joint_positions[:, parent_idx, :]
        + parent_rotation.apply(joint_positions[:, joint_idx, :]))
    global_joint_orientations[:, joint_idx, :] = (
        parent_rotation * local_rotation).as_quat()

```

The position calculation matters because a parent's rotation changes the direction of every child offset. The orientation calculation keeps the child's local rotation relative to its parent while producing a world-space orientation.

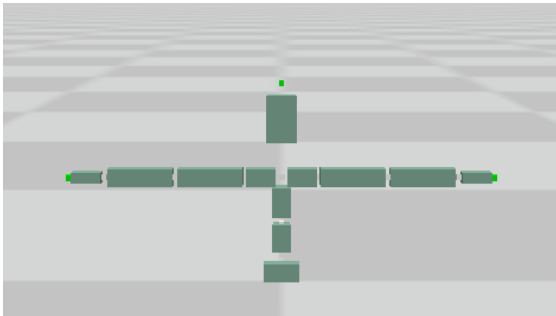


Figure 4. T-pose reconstructed from the joint hierarchy and offsets.

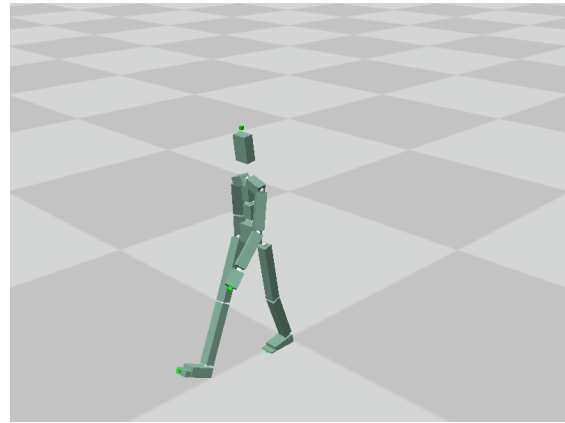


Figure 5. A pose from the forward-kinematics playback of the walking BVH.

Task 3 Inverse Kinematics

I completed the CCD update inside the designated code region of `task3_inverse_kinematic.py`. For each joint on the chain, I compared the direction from that joint to the current end effector with the direction from that joint to the target. Their cross product gives the rotation axis; the clipped dot product and `atan2` give a stable rotation angle.

```

vec_cur2end = chain_positions[end_idx] - chain_positions[current_idx]
vec_cur2target = target_pose - chain_positions[current_idx]
end_length = np.linalg.norm(vec_cur2end)
target_length = np.linalg.norm(vec_cur2target)
if end_length <= 1e-12 or target_length <= 1e-12:
    continue

vec_cur2end /= end_length
vec_cur2target /= target_length
dot = np.clip(np.dot(vec_cur2end, vec_cur2target), -1.0, 1.0)
axis = np.cross(vec_cur2end, vec_cur2target)
axis_length = np.linalg.norm(axis)

if axis_length <= 1e-12:
    if dot > 0.0:
        continue
    basis = np.array([1.0, 0.0, 0.0])
    if abs(vec_cur2end[0]) > 0.9:
        basis = np.array([0.0, 1.0, 0.0])
    axis = norm(np.cross(vec_cur2end, basis))
    angle = np.pi
else:
    axis /= axis_length
    angle = np.arctan2(axis_length, dot)

rot_vec = R.from_rotvec(angle * axis)
for joint_idx in range(current_idx, end_idx + 1):
    chain_orientations[joint_idx] = (
        rot_vec * chain_orientations[joint_idx])

```

I also guarded against zero-length vectors before normalizing them. This prevents undefined directions when the end effector is already at a joint or the target coincides with the joint. The explicit 180-degree case avoids relying on a zero cross product to define a rotation axis.

I compared the wrist position with a fixed target using several iteration settings. Starting from RootJoint, the wrist moved closer to the target as I allowed the solver more iterations. I also compared different chain starts while keeping the target and iteration setting the same. In this test, starting from either toe brought the wrist closer to the target than starting from RootJoint. The result depends on the starting pose and target, so I treat this as an observation from my test rather than a general rule.

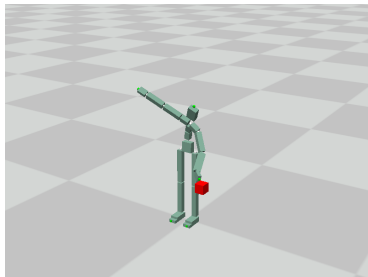


Figure 6. RootJoint-to-left-wrist chain with a smaller iteration budget.

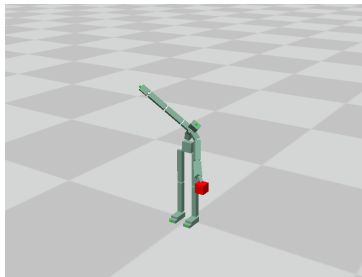


Figure 7. The same chain using the script's default iteration setting.

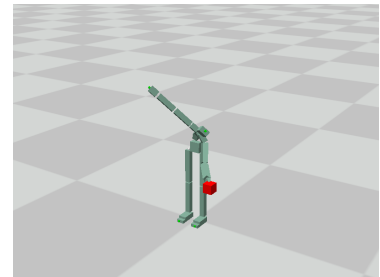


Figure 8. The same chain after allowing more iterations.

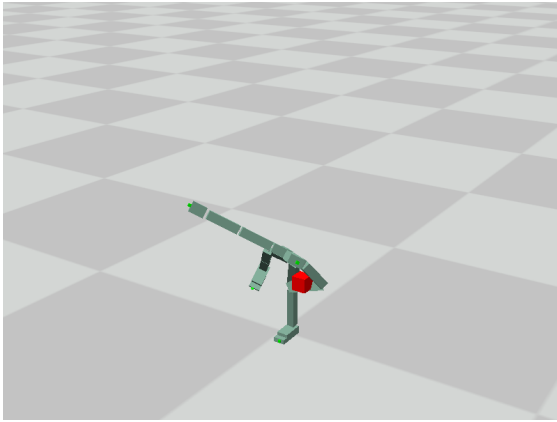


Figure 9. Left-wrist target test with ``lToeJoint_end`` as the named chain start.

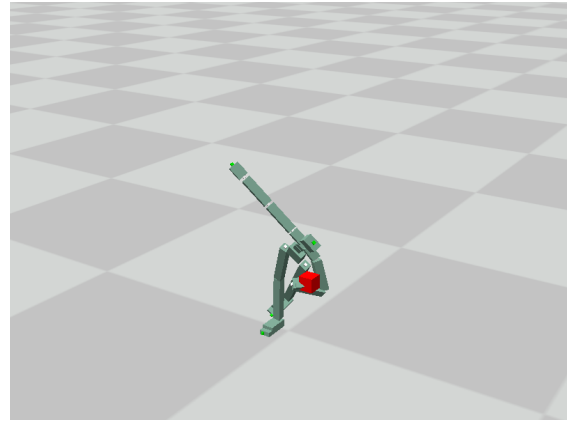


Figure 10. Left-wrist target test with ``rToeJoint_end`` as the named chain start.

What I Learned

- In Blender, a clean action and deliberately placed keyframes make it easier to build and review a new motion without confusing it with another animation. Previewing the whole timeline helped me judge the jump, wave, and repeated timing as a clip rather than as isolated poses.
- In forward kinematics, BVH offsets are local to each parent. To draw the full skeleton correctly, I must propagate both positions and orientations through the hierarchy; rotating an offset by the parent's global orientation is essential.
- CCD IK works by repeatedly reducing the end-effector-to-target distance. More iterations improved this particular result, but the chain choice also changed convergence. Degenerate vector cases need explicit handling to keep the rotation calculation stable.